rag-lesson-agent

Self-Evaluating Content Generation System | Architecture & Engineering Specification

Target Role: GenAI Engineer - Content Systems

Author: Vedant Tiwari (Full Stack / Security)

Production Models: Generator (Gemini 3.5 Flash) | Judge (Gemini 3.1 Flash Lite @ T=0)

Audit Checkpoints: 16 Binary Checks (5 Deterministic Code + 11 LLM)

1. System Architecture & Lifecycle Pipeline

The system executes an explicit, bounded **Finite State Machine (FSM)** designed to autonomously draft, audit, and regenerate educational materials for 12th-grade graduates from India with non-English backgrounds. Unlike unconstrained conversational chains, our pipeline isolates authoring authority from evaluation authority, enforces hard PASS/FAIL gates, and maintains deterministic human-in-the-loop (HITL) memory safety.

Lifecycle Phase	Agent & Operational Logic	I/O & Boundary Contracts
Phase 1: Planning agents/planner.py	Analyzes topic, ingests grounding facts, profiles Indian 12th-pass learner, and selects single pedagogical analogy (The Open-Book Exam).	**Input:** knowledge/rag_facts.md **Output:** output/lesson_spec.json
Phase 2: Drafting agents/generator.py	Drafts lesson in simple English with short sentences (avg ≤ 15 words, max 25 words). Explains jargon inline at first use. Integrates plain English glossary.	**Model:** gemini-3.5-flash (T=0.7) **Output:** output/attempts/attempt_N.md
Phase 3: Dual Audit agents/evaluator.py	Executes 16 hard PASS/FAIL checks across 2 layers: • **Layer 1 (Code):** Words (B1,B2), Jargon (B3a), Headings (D1), Quiz (E2a) • **Layer 2 (LLM):** 11 semantic checks (A1..E2b) with verbatim evidence validation.	**Model:** gemini-3.1-flash-lite (T=0) **Rubric:** rubric/rubric.yaml **Result:** 16/16 Binary Array
Phase 4a: Shipped main.py (Router)	Triggered when all 16 checks PASS. Marks run as SHIPPED, logs summary metrics, and notifies the client.	**Destination:** output/final_lesson.md output/run_summary.json
Phase 4b: Retry Loop Bounded Retries (< 3)	Packages failed check IDs, exact failure reasons, verbatim quotes, and actionable fix suggestions directly back to Generator.	**Feedback:** Surgical revision payload. **Log:** logs/rejection_log.jsonl
Phase 4c: Quarantine Exhausted Retries (= 3)	Halts pipeline without shipping flawed content. Persists diagnostic failure report for human inspection.	**Quarantine:** output/NEEDS_HUMAN_REVIEW.md
Phase 5: Reflection agents/reflector.py	Aggregates failure metrics across runs. Synthesizes operational generator rules after ≥ 2 failures. Quarantines rubric modifications for human authorization.	**Memory:** failure_stats.json **Rules:** learned_rules.json **Rubric:** proposed_rubric_changes.json

2. Deep Dive: Component Architectures

A. Planner Agent (Curriculum Architecture)

The Planner anchors the session before drafting begins. It ingests strict grounding facts from knowledge/rag_facts.md, defines essential core concepts (Retrieval, Augmented, Generation, Two-Phase Execution, Model Weights Immutability), and selects a single pedagogical analogy. For RAG, it mandates *The Open-Book Exam* (LLM = student, retrieval = browsing notes, generation = writing answer). It outputs a machine-readable JSON specification to output/lesson_spec.json.

B. Generator Agent (Constrained Drafting & Targeted Revision)

Powered by gemini-3.5-flash (T=0.7), the generator adheres to strict lexical and pedagogical rules: 1) Sentences must average ≤ 15 words with a hard ceiling of 25 words; 2) Every technical jargon term (LLM, chunk, embedding, vector database) must be defined inline on first occurrence; 3) Explicit prohibition against claiming RAG stops or eliminates hallucinations 100% (it only reduces them); 4) Surgical retry feedback loop: On Attempt > 1, it receives only failed check IDs, reasons, and quotes, revising failing blocks while leaving validated sections untouched.

C. Dual-Layer Evaluator (16 Hard PASS/FAIL Checks)

The evaluator combines deterministic code verification with zero-temperature LLM reasoning to ensure zero tolerance for errors:

Check ID	Type	Target Quality Criteria	Enforcement Layer
A1	LLM	Correct RAG definition as retrieval + prompt augmentation + LLM generation	gemini-3.1-flash-lite (T=0)
A2	LLM	Grounding Integrity: Zero fact contradictions (weights never change, no 100% hallucination claims)	Grounding Fact Sheet Auditor
A3	LLM	Mandatory Limitations: Retrieval misses, latency overhead, bad notes risk	Zero-Temp LLM Judge
B1	CODE	Average sentence length ≤ 15 words across full markdown text	Python regex sentence tokenizer
B2	CODE	Hard maximum sentence ceiling ≤ 25 words (zero runaway sentences)	Python sentence length validator
B3a	CODE	Jargon defined at first occurrence using is/means/(...) heuristics	Deterministic Regex Lexicon
B3b	LLM	Jargon explanations use plain, accessible beginner terminology	Zero-Temp LLM Judge
B4	LLM	Absence of confusing Western idioms, slang, or academic posturing	Vocabulary Auditor
C1	LLM	Single consistent analogy sustained from hook to recap	Analogy Continuity Judge
C2	LLM	Concrete worked example (User Question, Numbers/Vectors, Retrieved Chunk, Final Answer)	Pedagogical Example Auditor
D1	CODE	Required H2 structural headings: What, Why, and How	Markdown AST Parser
D2	LLM	Two-Phase Pipeline: Phase 1 (Prep: chunk + embed documents in advance) vs Phase 2 (Query time: embed question + retrieve + answer)	Two-Phase Pipeline Enforcer
D3	LLM	Why-it-matters core benefits: Hallucination mitigation + Outdated cutoff + Private data	Concept Coverage Judge
E1	LLM	Logical teaching flow: What -> Why -> Two-Phase How -> Worked Example -> Limits -> Recap	Pedagogical Flow Judge
E2a	CODE	Recap section present and exactly 3 numbered quiz questions (Q1, Q2, Q3)	Regex Quiz Parser (excluded)
E2b	LLM	All 3 quiz questions unambiguously answerable from the lesson text alone	Quiz Grounding Judge

D. Memory, Self-Reflection & Human Guardrails

Asymmetric Learning Security: An autonomous agent must never possess write access to its own examination rubric. The Reflector (agents/reflector.py) tracks cumulative failures in memory/failure_stats.json. When a check fails repeatedly (≥ 2 times), it synthesizes an operational rule for the Generator. However, rules are marked with an approved: false flag until explicitly validated by a human operator (HITL). Any suggestion to modify the rubric is quarantined into memory/proposed_rubric_changes.json and never executed automatically.

3. Enterprise Security & Web Architecture

To satisfy production security standards, the FastAPI web layer (web/app.py, web/security.py) implements 13 defense-in-depth security mitigations documented in SECURITY.md:

Threat Vector	Engineering Defense Mechanism	Implementation File
CSRF Attacks	Double-submit cookie pattern with cryptographic HMAC constant-time matching and automatic cookie/header resync.	web/security.py
XSS (Cross-Site Scripting)	Server-side tag stripping combined with client-side DOMPurify sanitization before rendering marked.js markdown.	web/static/app.js
Content Security Policy (CSP)	Strict CSP headers: default-src 'self', zero external CDN scripts, vendored marked/purify assets.	SecurityHeadersMiddleware
Rate Limiting & DoS	Sliding-window IP rate limiting (120 req/min, 20 runs/hour, max 3 concurrent background generation tasks).	RateLimiter
Prompt Injection	User topics and drafts wrapped in strict structural delimiters (<UNTRUSTED_CONTENT>); instructions ignored.	prompts/judge_v1.md
Credential Leaks	Automated regex secret redactor scrubbing AIza[0-9A-Za-z_-]{30,45} and GEMINI_API_KEY from logs & traces.	llm.py (redact_secrets)

4. Strategic System Enhancements & Future Roadmap

To evolve rag-lesson-agent from a single-node assessment system into an enterprise-scale content generation engine, the following architectural expansions are planned:

1. Cross-Family Multi-Model Jury (Eliminating Family Bias)

Current State: Both generator (Gemini 3.5 Flash) and judge (Gemini 3.1 Flash Lite) share Google's underlying model architecture and training priors.
Roadmap: Implement a multi-model jury voting protocol. For critical checks (A1 Definition, A2 Grounding, D2 Pipeline Order), the system queries an ensemble: Gemini 3.1, Anthropic Claude 3.5 Haiku, and an open-weights model (e.g. Llama-3.3-70B via Ollama/vLLM). A 2-out-of-3 consensus determines the verdict, eliminating shared cognitive blind spots.

2. Vector Database & Live Dynamic Retrieval Grounding

Current State: Facts are loaded from a static markdown file (knowledge/rag_facts.md).
Roadmap: Integrate an embedded vector database (e.g. ChromaDB or Qdrant). Grounding facts, recent curriculum guidelines, and pedagogical research papers will be embedded dynamically using gemini-embedding-001. The planner can query authoritative educational repositories dynamically based on the requested subject.

3. Automated EvalOps & Precision/Recall Benchmark Harness

Current State: Evaluator quality is validated against 5 deliberate bad lessons in tests/bad_lessons/.
Roadmap: Build a synthetic benchmark harness generating 100+ annotated good and bad lessons spanning subtle flaws (e.g. subtle order inversion, mild jargon leaks, borderline sentence length). Automatically compute **Precision, Recall, and F1-Score** for every rubric check on each git commit, alerting if prompt drift impairs error-catching recall.

4. Direct Preference Optimization (DPO) Dataset Curation

Current State: Lessons are drafted and rejected in logs/rejection_log.jsonl.
Roadmap: Utilize the rejection logs as an automated source of paired preference data: *rejected draft* (Attempt 1) vs *accepted draft* (Attempt 2). Export this data to standard DPO (Direct Preference Optimization) format to fine-tune smaller, cost-effective edge models (e.g. Gemma-2-9B) to draft compliant educational content on Attempt 1 with zero retries.

5. Multilingual Indian Vernacular Translation Engine

Current State: The lesson is drafted in simple English with an English glossary.
Roadmap: Add a downstream localization agent that takes the validated, shipped lesson and translates it into Hindi, Tamil, Telugu, Marathi, and Bengali while preserving code blocks, ASCII diagrams, and interactive quiz formatting.

6. Production Observability & Distributed Tracing

Current State: Logs are stored in local JSON Lines (logs/rejection_log.jsonl) and console streams.
Roadmap: Integrate OpenTelemetry instrumentation connected to Langfuse, Arize Phoenix, or Datadog. Track token usage, per-check LLM latency, cost per shipped lesson, and time-to-ship across thousands of concurrent student sessions.

Document Status: Verified & Finalized | **Repository:** github.com/askvedant18/rag-lesson-agent | **Branch:** main